

OS SEIS ELEMENTOS “DEFINITÓRIOS” DE UMA ARQUITETURA

0 INTRODUÇÃO

1. OS REGISTRADORES ACESSÍVEIS AO PROGRAMADOR (*EM ASSEMBLY LANGUAGE*)
2. O CONJUNTO DE INSTRUÇÕES
3. OS FORMATOS DE INSTRUÇÃO
4. OS MODOS DE ENDEREÇAMENTO
5. A LINGUAGEM DE MONTAGEM (*ASSEMBLY LANGUAGE*)
6. O MODELO DE ACESSO À MEMÓRIA

Última Atualização em 23/07/2026



0 – INTRODUÇÃO

INSTRUCTION SET ARCHITECTURE - ISA

- Uma *arquitetura* → é uma abstração de um (ou de uma classe de) processador(es)
 - Nome completo: Arquitetura do Conjunto de Instruções (em inglês *Instruction Set Architecture* ou ISA)
- *Dominar* uma ISA → saber programar o processador que ela descreve (em linguagem de montagem)
- Arquitetura **NÃO** é Organização
 - E.g. – dezenas (centenas?) de organizações diferentes usam a arquitetura ARM v8!

0 – INTRODUÇÃO

ORGANIZAÇÃO X ARQUITETURA

- **Organização de Processadores** – A visão abstrata do engenheiro (elétrico, de computação) de um computador
 - Descrição contendo **transistores** e/ou **portas lógicas** e/ou **registradores** e/ou **unidades lógico-aritméticas**, **fios**, **multiplexadores**, **barramentos** etc.
 - Alguns Autores denominam **Organização** usando o termo **Micro-Arquitetura**, termo que o Autor acha ruim, mas muitos adotam esta nomenclatura...
- **Arquitetura de Processadores** – A visão abstrata do programador de baixo nível (linguagem de montagem, em inglês *assembly language*)
 - Descrição em termos de: **instruções** que o processador executa, **registradores** que o programador enxerga e usa para armazenar dados, a **linguagem de programação de montagem** etc.

0 – INTRODUÇÃO

DEFINIÇÃO DE PROCESSADOR

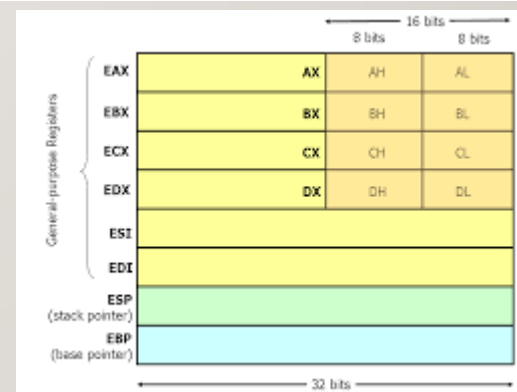
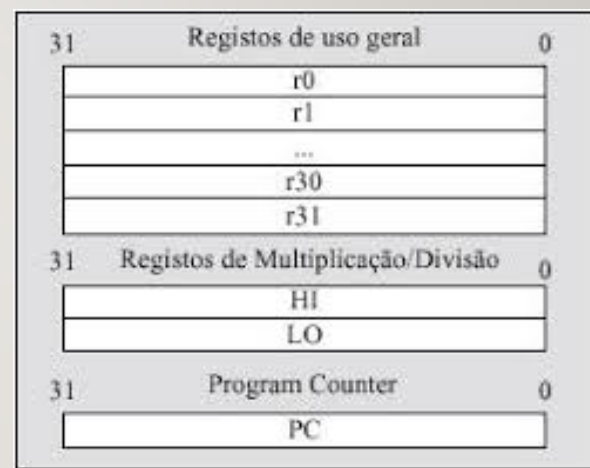
- Mas afinal, o que é um **processador** (ou computador)? Uma definição seria
 - Uma máquina com capacidade de acesso a meios de armazenamento onde estão estocadas informações a serem processadas e as informações que dizem como processar as primeiras. Também se chama o processador principal de um computador de **CPU** (do inglês *Central Processing Unit*)
 - As informações a serem processadas – são os dados
 - As informações de como processar dados – são os programas
 - **Programas** – sequências de *instruções*, retiradas de um conjunto fixo de instruções reconhecidas como tal pelo processador
 - **Funcionamento de um processador**: repetir, infinitamente, a **sequência de 3 ações**: (1) buscar instrução, (2) identificar a instrução buscada, (3) executar a instrução buscada
 - A execução de instruções pode incorrer em acesso a dispositivos de entrada e saída

0 – INTRODUÇÃO

- Alguns exemplos de ISAs usados hoje em dia → há dezenas, talvez centenas!!
 - x86 → arquitetura definida pela Intel e fabricada por ela, pela AMD etc.
 - ARM v8 → arquitetura definida pela ARM e usada pela Samsung (no Galaxy S10), pela Apple (no iPhone 11) etc
 - MIPS32 e MIPS64 → usadas pela Mobileye (Cia. da Intel para sensores automotivos), pela Tesla e pela Volvo em automóveis, em roteadores WiFi (da Cisco, etc.), aplicações em IA etc
 - AVR → Criada pela Atmel (da Microchip Tech desde 2016), usada em kits Arduino, em carros da BMW e Daimler-Chrysler, no controle do Xbox...
 - MicroBlaze e Nios, propostas para uso em FPGAs (Xilinx e Altera, resp.)
 - RISC-V → Proposta na UC Berkeley → classe de ISAs abertas implementáveis (sem *royalties*) por qquer pessoa/empresa. Google usa no Titan-M2 (segurança), em uso por múltiplas empresas (China, Índia etc)

I – OS REGISTRADORES ACESSÍVEIS AO PROGRAMADOR (EM LINGUAGEM DE MONTAGEM)

- Registradores → um dos elementos mais importantes do processador
- Registradores → visão de Organização (Hw): conjunto de flip-flops com mesmo clock
- Registradores → visão Arquitetura (ISA): armazenadores de informação (dados, controle etc.)
- Registradores → dentro do processador (memórias → fora)
- Instruções → manipulam dados de registradores (tipicamente)
- Existem ISAs com apenas 1 registrador (pode haver mais em organizações específicas da ISA)
- Existem ISAs com 2, 4, 8, 16, 32, 128 ou até mais registradores
 - Exemplo 1: MIPS-I possui 35 regs (32 genéricos + 2 para Mult/Div + PC)
 - Exemplo 2: x86 possui 8/16 registradores



2 – O CONJUNTO DE INSTRUÇÕES

- Instruções → o elemento “definitório” mais importante do processador
- Visão arquitetural (ISA): uma instrução → sequência de *operações* atômicas que alteram registradores e/ou memória
- Cada ISA: um número fixo de instruções (14, 36, 536, varia muito e há exceções...)
- Instruções classificáveis segundo vários critérios (função, tamanho, duração, no. de operandos etc.)
- Exemplo: Uma parte do conjunto de instruções do MIPS-I →

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Three operands; data in registers
	subtract	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	Three operands; data in registers
Data transfer	load word	lw \$s1,100(\$s2)	$\$s1 = \text{Memory}[\$s2 + 100]$	Data from memory to register
	store word	sw \$s1,100(\$s2)	$\text{Memory}[\$s2 + 100] = \$s1$	Data from register to memory
Logical	and	and \$s1,\$s2,\$s3	$\$s1 = \$s2 \& \$s3$	Three reg. operands; bit-by-bit AND
	or	or \$s1,\$s2,\$s3	$\$s1 = \$s2 \mid \$s3$	Three reg. operands; bit-by-bit OR
	nor	nor \$s1,\$s2,\$s3	$\$s1 = \sim (\$s2 \mid \$s3)$	Three reg. operands; bit-by-bit NOR
	and immediate	andi \$s1,\$s2,100	$\$s1 = \$s2 \& 100$	Bit-by-bit AND reg with constant
	or immediate	ori \$s1,\$s2,100	$\$s1 = \$s2 \mid 100$	Bit-by-bit OR reg with constant
	shift left logical	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$	Shift left by constant
	shift right logical	srl \$s1,\$s2,10	$\$s1 = \$s2 \gg 10$	Shift right by constant
Conditional branch	branch on equal	beq \$s1,\$s2,L	if ($\$s1 == \$s2$) go to L	Equal test and branch
	branch on not equal	bne \$s1,\$s2,L	if ($\$s1 \neq \$s2$) go to L	Not equal test and branch
	set on less than	slt \$s1,\$s2,\$s3	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than; used with beq, bne
	set on less than immediate	slt \$s1,\$s2,100	if ($\$s2 < 100$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than immediate; used with beq, bne
Unconditional jump	jump	j L	go to L	Jump to target address

3 – OS FORMATOS DE INSTRUÇÃO

- Exemplo introdutório

Um exemplo de programa escrito em linguagem de montagem do MIPS

```
ADDIU    $29, $29, -32
SW       $31, 20($29)
SW       $4 , 32($29)
SW       $5 , 36($29)
SW       $0 , 24($29)
SW       $0 , 28($29)
LW       $14, 28($29)
LW       $24, 24($29)
MULTU    $14, $14
ADDIU     $8 , $14, 1
SLTI      $1 , $8, 101
SW        $8 , 28($29)
MFLO      $15
ADDU      $25, $24, $15
BNE       $1 , $0, -9
SW        $25, 24($29)
LUI       $4 , 4096
LW        $5 , 24($29)
JAL       1048812
ADDIU     $4 , $4, 1072
LW        $31, 20($29)
ADDIU     $29, $29, 32
JR        $31
MOVE      $2 , $0
```

O código-objeto do programa à esquerda, pronto para ser executado se carregado em memória

```
0010011111011111011111111111111100000
101011111101111111000000000000010100
10101111110100100000000000000100000
10101111110100101000000000000100100
1010111111010000000000000000011000
1010111111010000000000000000011100
1000111111010111000000000000011100
1000111111011100000000000000011000
00000001110011100000000000011001
001001011100100000000000000000001
00101001000000010000000001100101
1010111111010100000000000000011100
0000000000000000000111100000010010
00000011000011111100100000100001
00010100001000001111111111110111
1010111111011100100000000000011000
00111100000001000001000000000000
1000111111010010100000000000011000
000011000001000000000000011101100
00100100100001000000010000110000
1000111111011111100000000000010100
0010011111011111010000000000100000
00000011111000000000000000001000
0000000000000000000001000000100001
```

- Pergunta: Como o texto à esquerda se traduz nos bits à direita?
- Resposta: Através dos formatos de instrução!

3 – OS FORMATOS DE INSTRUÇÃO

- Tomemos o código fonte da primeira instrução: **ADDIU \$29, \$29, -32**
- O formato da instrução definida por **ADDIU**, retirado do manual de instruções do MIPS é:

Adição com dado imediato (sem overflow)

Formato Fonte:

addiu rt, rs, imm

Formato Objeto:

9	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Semântica: Coloca a soma do registrador rs e do dado imediato (imm) com sinal estendido no registrador rt.

- Com esta informação é possível gerar o código objeto a partir do código fonte, pois
 - A constante 9 em binário representada em 6 bits é **001001**
 - O número do registrador rt (**\$29**) em 5 bits é **11101**. rs é o mesmo registrador **\$29**
 - **-32**, representado em complemento de 2 em 16 bits é: **111111111100000**
 - Unindo-se estes 4 valores, obtém-se o código objeto da instrução, que é
 - **0010011110111101111111111100000**, que representado de forma mais compacta, em hexadecimal é **0x27BDFFE0**
 - Na língua do MIPS, **0x27BDFFE0** significa então: “Some -32 ao conteúdo do registrador \$29 e armazene o resultado no registrador \$29”

3 – OS FORMATOS DE INSTRUÇÃO

- Agora pode-se generalizar a discussão de formatos de instrução
 1. Formatos de instrução estabelecem uma relação unívoca entre o que o programador escreve (em linguagem de montagem) e o código executável pelo processador
 2. Cada instrução tem associado um nome mnemônico e um conjunto de operandos
 3. Operandos podem ser referências a registradores, a constantes, referências a posições de memória, referências indiretas, etc. O modo de endereçamento associado a cada operando determina como obter o operando durante a execução da instrução
 4. Cada instrução pode ter um ou mais formatos associados a ela. Em máquinas RISC, frequentemente cada instrução possui um único formato
 5. A parte constante do formato (por exemplo o 9 no campo de 6 bits mais à esquerda da ADDIU, denominado de *opcode* ou código da operação, ver slide anterior) determina a instrução, enquanto que a parte variável (rs, rt, imm na ADDIU) determina um ou mais operandos desta

4 – OS MODOS DE ENDEREÇAMENTO

- Define-se modos de endereçamento como “a forma de recuperar cada operando de uma instrução”
- Exemplo → a instrução **BNE \$1, \$0, -9** do exemplo de programa MIPS apresentado antes possui 3 operandos: \$1, \$0 e -9. O caractere \$ identifica que os dois primeiros operandos são referências a registradores do processador. Se observarmos o formato desta instrução no manual, tem-se

Salta (branch) se for diferente

Formato Fonte:

bne rs, rt, imm

Formato Objeto:

5	rs	rt	offset
6 bits	5 bits	5 bits	16 bits

Semântica: Se o registrador rs contiver valor diferente do valor armazenado no registrador rt, pule o número de instruções designado pelo *offset* (para a frente se o *offset* for positivo, para trás se o *offset* for negativo), a partir da instrução que segue a instrução bne

- Os três operandos desta instrução usam dois modos de endereçamento distintos. Os dois primeiros (\$1 e \$0) usam o **modo a registrador**, pois os bits dos campos rs e rt representam números de registradores que contêm dados a usar na execução do bne. O último operando usa o **modo relativo** de endereçamento, o operando é o deslocamento (*offset*) em instruções para chegar à próxima instrução a ser executada, em relação à instrução seguinte ao bne, caso a comparação dos conteúdos dos registradores revele que estes são diferentes

4 – OS MODOS DE ENDEREÇAMENTO

- Os modos de endereçamentos mais comumente encontrados na grande maioria das arquiteturas são os seguintes
 - **Modo Imediato** → o operando é uma constante contida dentro do código da instrução, estando assim *imediatamente* disponível para uso, o que justifica o nome do modo
 - **Modo a Registrador** → o operando é uma referência a (i.e. um endereço de) um registrador interno do processador
 - **Modo Direto** ou **Absoluto** → o operando é uma referência a (i.e. um endereço de) uma posição da memória externa ao processador (e.g. não existe no MIPS)
 - **Modo Relativo** → o operando é um valor a ser adicionado a um registrador existente para obter o endereço de um operando (no exemplo do MIPS, o modo relativo é relativo ao valor do registrador *program counter*, PC, do MIPS)
 - **Modo Indireto** → O operando é o endereço do endereço de um dado (muito útil para manipular vetores e matrizes em linguagem de montagem)
 - **Modo Base-Deslocamento** → Usa-se aqui dois valores para permitir localizar o operando: o primeiro é um endereço base (em um registrador ou em uma posição de memória), o segundo é uma constante a ser somada com o endereço base para obter o endereço do operando da instrução

4 – OS MODOS DE ENDEREÇAMENTO

- Observações
 - I. Cuidado com documentação de fabricantes
 - De forma confusa algumas destas costumam associar modos de endereçamento a *instruções* e não a cada *operando* desta, como deveriam
 - Por exemplo, uma instrução ADDIU \$29, \$29,-32 seria normalmente classificada, em tais documentos, como uma instrução que usa endereçamento imediato, ignorando que os dois primeiros operandos usam o modo de endereçamento a registrador. Isto é péssimo e confuso, mas é a vida real...

5 – A LINGUAGEM DE MONTAGEM (ASSEMBLY LANGUAGE)

- Trata-se de uma **linguagem de programação**
- Existe uma **linguagem de montagem** diferente para cada **arquitetura (ISA)**
- Como em qualquer linguagem existem **comandos** → as **instruções** da **arquitetura**
- Como em qualquer linguagem existem **dados** → basicamente **números** de várias classes, **textos** (caracteres e/ou cadeias de caracteres), e **menções** a conteúdos de **estruturas específicas**, como **registradores**
- Além de comandos e dados, existem **diretivas de montagem** → servem para definir dados, alinhamento de código, macros etc.
- Como em poucas linguagens, pode-se referenciar registradores (específicos) da arquitetura (e de qualquer organização desta) em programas
- **Programas** são **sequências de instruções**
- Instruções possuem formatos próprios (mnemônicos, sintaxe de operandos etc.)
- A seguir, dá-se um exemplo de um programa completo com comentários, diretivas de montagem, sequência de instruções e definições de dados

```
#####
# Exemplo de programa completo em linguagem de montagem do MIPS : Manipulando vetores
# Autor: Ney Calazans
# Função: Este programa soma o valor de uma constante (const) a cada elemento do vetor array
#####
```

```

        .text                # Diretiva para o montador - adiciona o que vem abaixo
                                #      à memória de programa do processador
        .globl  main         # Diretiva - Declara o rótulo main como sendo global
                                #      main é o ponto a partir de onde se inicia a execução

main:
        la      $t0,array    # Pseudo: O registrador $t0 contém o endereço do vetor array
        la      $t1,size     # Obtém o endereço da posição da memória de dados onde se guarda
                                #      o tamanho do vetor
        lw      $t1,0($t1)    # O registrador $t1 carregado com o tamanho/nro elementos do vetor a tratar
        la      $t2,const     # Obtém o endereço da constante const
        lw      $t2,0($t2)    # O registrador $t2 contém a constante a somar
loop:    blez     $t1,end      # Se o elementos a tratar é (ou chega a) 0, fim do processamento
        lw      $t3,0($t0)    # Senão, busca na memória o próximo elemento do vetor
        addu     $t3,$t3,$t2   # Soma o elemento atual com a constante
        sw      $t3,0($t0)    # Atualiza o vetor em memória
        addiu    $t0,$t0,4     # Faz o apontador do vetor ($t0) apontar para o próximo elemento
                                #      lembrar que 1 palavra no MIPS ocupa 4 endereços consecutivos de memória
        addiu    $t1,$t1,-1    # Decrementa o número de elementos que resta tratar do vetor
        j        loop         # Volta a executar novamente o teste de permanência no laço
                                #      Ao final do laço, volta-se para o Sistema Operacional (SO)
end:    li  $v0,10            # Prepara chamada do sistema para finalizar programa
        syscall              # Instrução de chamada do SO - Produz o fim do programa

        .data                # Diretiva para o montador - adiciona o que vem abaixo
                                #      à memória de dados do processador
array:   .word    0x12 0xff 0x3 0x14 0x878 0x31 0x62 0x10 0x5 0x16 0x20 # O vetor de dados em si
                                #      A diretiva .word reserva 4 bytes (uma palavra) para cada inteiro,
                                #      em posições sucessivas da memória de dados
size:    .word    11          # Variável que armazena o tamanho do vetor (no caso, 11 elementos)
const:   .word    0x100       # Constante a somar a cada elemento do vetor

```

6 – O MODELO DE ACESSO À MEMÓRIA

- Fora do processador existe apenas a **Memória** (visão da **Arquitetura**)
- Processador acessa a **Memória** lendo ou escrevendo nela
- Endereço – valor em **m** fios, define que a **Altura** da memória é 2^m
- **Largura** da memória **n** – unidade de acesso mínimo à memória
- Dados – Conjunto de **p** fios que trazem/levam dados de/para a **memória**. Note-se **p** pode ser igual a **n** ou diferente de **n**
→ **p** costuma ser múltiplo de **n**
- Mapa de memória – Uma abstração → define a memória como uma tabela de $2^m \times n$ bits

